

StockWise

Smart Inventory & Stock Tracking System Installation & Deployment Guide

1. Introduction

This document provides step-by-step instructions for setting up and deploying the StockWise application in a local development or production environment. Following this guide ensures that all components — Backend (Spring Boot), Frontend (React.js), and Database (PostgreSQL) — are properly configured and connected.

2. Prerequisites

Before beginning the installation, ensure the following software is installed on your machine:

Software	Version	Purpose
Java JDK	21 or higher	Backend runtime environment
Apache Maven	3.8+	Build and dependency management
Node.js & npm	18 LTS or higher	Frontend build toolchain
PostgreSQL	15 or higher	Relational database server
Git	Latest stable	Version control and cloning
IDE (IntelliJ/VSCode)	Any recent version	Recommended development environment

3. Repository Setup

Clone the project repository from GitHub to your local machine:

```
git clone https://github.com/stockwise-team/stockwise.git
cd stockwise
```

The repository structure is organized as follows:

- stockwise/backend — Spring Boot application source code
- stockwise/frontend — React.js application source code
- stockwise/docs — Project documentation and UML diagrams
- stockwise/scripts — Database initialization scripts

4. Database Configuration

4.1 Create PostgreSQL Database

Open the PostgreSQL command line or pgAdmin and execute the following:

```
CREATE DATABASE stockwise_db;  
CREATE USER stockwise_user WITH PASSWORD 'your_password';  
GRANT ALL PRIVILEGES ON DATABASE stockwise_db TO stockwise_user;
```

4.2 Configure application.properties

Navigate to backend/src/main/resources/application.properties and update the database credentials:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/stockwise_db  
spring.datasource.username=stockwise_user  
spring.datasource.password=your_password  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```



Note: On first startup, Spring Boot will automatically create the required tables via JPA/Hibernate based on the entity models. No manual schema script execution is needed.

5. Backend Installation & Startup

Navigate to the backend directory and build the project using Maven:

1. Open a terminal and navigate to the backend folder:

```
cd stockwise/backend
```

2. Install dependencies and build:

```
mvn clean install
```

3. Run the Spring Boot application:

```
mvn spring-boot:run
```

4. Verify the backend is running by accessing:

```
http://localhost:8080/api/health
```



The backend server runs on port 8080 by default. This can be changed in application.properties using: server.port=<your_port>

6. Frontend Installation & Startup

Navigate to the frontend directory and install the required Node.js dependencies:

5. Open a new terminal and navigate to the frontend folder:

```
cd stockwise/frontend
```

6. Install npm packages:

```
npm install
```

7. Configure the backend API URL in the .env file:

```
REACT_APP_API_BASE_URL=http://localhost:8080/api
```

8. Start the development server:

```
npm start
```

9. The application will open automatically at:

```
http://localhost:3000
```

7. Default Login Credentials

Upon first launch, use the following default credentials to access the system:

Role	Username	Password	Access Level
Admin	admin	admin123	Full Access
Staff	staff1	staff123	Limited Access

! Important: Change the default passwords immediately after first login for production environments.

8. Production Deployment

8.1 Building for Production

To deploy StockWise to a production server, follow these steps:

10. Build the backend JAR file:

```
cd backend && mvn clean package -DskipTests
```

11. Build the optimized frontend bundle:

```
cd frontend && npm run build
```

12. Copy the build/ folder to your web server (e.g., Nginx or Apache).

13. Deploy the backend JAR on the server:

```
java -jar target/stockwise-1.0.0.jar
```

8.2 Environment Variables for Production

- Set `SPRING_DATASOURCE_URL` to the production database URL.
- Set `SPRING_DATASOURCE_USERNAME` and `SPRING_DATASOURCE_PASSWORD` securely.
- Configure `REACT_APP_API_BASE_URL` to the production backend domain.
- Set `JWT_SECRET` to a strong, randomly generated secret key.

9. Troubleshooting

Common Issue	Resolution
Cannot connect to database	Verify PostgreSQL is running and credentials in <code>application.properties</code> are correct.
Port 8080 already in use	Change <code>server.port</code> in <code>application.properties</code> or stop the conflicting process.
npm install fails	Delete <code>node_modules/</code> and <code>package-lock.json</code> , then run <code>npm install</code> again.
CORS errors in browser	Ensure CORS is configured in the Spring Boot <code>WebConfig</code> class for the frontend URL.
Tables not created on startup	Set <code>spring.jpa.hibernate.ddl-auto=create</code> on first run, then revert to <code>update</code> .